

What We Built Today — Explained Super Simply

Day: 18 July 2026 **For:** Omar (DEV-1) **Promise:** every big word is explained like you are 5 — but nothing is skipped.

First, the story (so everything else makes sense)

Imagine we are building a **magic toy store**. But it is special: **many different shop-owners can each have their OWN store inside it, all at the same time**, and none of them can see each other's toys. That is what "multi-tenant" means — many *tenants* (shop-owners) share one big building.

Here are the toys and words we use every day:

Word we say	What it really is (baby words)
Odoo	A giant, ready-made LEGO castle kit . We build our store ON TOP of it instead of making every brick ourselves.
Database (Postgres)	The store's memory box . It remembers every toy, price, and person.
The real store	The actual shop, at door number 8069 . Real toys live here.
The demo	A pretend cardboard shop , at door number 5173 . Most toys are fake — it's just to <i>show</i> people what the real store will look like.
An "issue"	One job on our to-do list . We have 100 jobs, in order.
A "branch"	Your own little building table , so you don't bump the big castle while you build.
A "PR"	Raising your hand: <i>"Please add my piece to the big castle."</i>
The 4 robot checkers (CI)	Four robots that check your piece before it joins.
"Merge"	Your piece clicks into the big castle.
develop / main	develop = the messy workbench where we add new pieces. main = the clean display shelf (the safe, finished copy).

Now, here is everything we finished today. We did **5 big builds**. 🧱

Build 1 — We made the pretend shop's front door REAL 🧱

(Job #104 — "Wire demo login & signup to the Odoo backend")

Before: The pretend cardboard shop had a **fake door**. You clicked "Sign in" and it just *pretended* to let you in. No real checking.

What we did: We connected the pretend shop's **door** (only the door!) to the **real memory box**. Now:

- When you type your name and secret word, the real store **actually checks** if you are allowed in.
- If you type the **wrong** secret word, it says "no, that's wrong" (for real).
- You can **make a brand-new account**, and a **real person-card gets saved in the memory box**.

The tricky part (not skipped): The pretend shop lives at door 5173 and the real store lives at door 8069. Browsers get scared when two different doors talk (they think it's stealing). So we built a secret **hallway** (a "proxy") that makes the two doors *look* like the same door, so they are allowed to talk and share the "you're logged in" sticker (a cookie).

We also learned 3 secret Odoo rules the hard way (wrote them down so we never trip again): the new-account door only works with a special header, and sending that same header to the *login* door makes Odoo **forget** to give the sticker. Only the pretend toys are still fake — the **door is 100% real**.

Build 2 — We made a magic "start a job" button ✨

(Job #105 — the `/solve-issue` helper + the memory file)

The problem: Every time we start a new job in a fresh chat, I (Claude) **forget everything** — like waking up with no memory. And we might start a job **too early**, before the jobs it depends on are done.

What we built — two things:

1. **A magic button called `/solve-issue`**. You type `/solve-issue 3` and it **automatically checks 7 things** before writing ANY code:
 - Is this job still open (not already done)?
 - Is it a real job on our list?
 - Is it a "wait until later" job? (some jobs are frozen for now)
 - **Are all the jobs it needs finished FIRST?** ← the most important check
 - Is anyone else already doing it?
 - Is the workbench clean and the store turned on?

If something is wrong, it **stops** and gives you choices — it never barrels ahead.

2. **A memory file called `CLAUDE.md`**. It loads **automatically every time**, so I never wake up with no memory again. It has the whole store map inside it.

We even tested the magic button both ways to be sure: on a job with no waiting → it said **GO** ✅. On a job that was waiting for another → it said **STOP** and named exactly what to finish first ✅.

Build 3 — We finished two empty rooms and added smoke alarms 🔔

(Job #102 = P1-T01 — "Addon Skeleton & Test Scaffolding")

Our store is made of **rooms** (in grown-up words: "modules/addons"). Two rooms (`ncollection_core` and `ncollection_saas`) were **totally empty** — just bare walls.

What we did:

- Built the **standard shelves** inside both empty rooms (folders named `models`, `views`, `security`, `tests`, and more) so any builder walking in knows exactly where things go.
- Put a tiny **smoke alarm** in each room (a "smoke test"). It's a little checker that goes *beep* — yes, *this room works!* every time the robots inspect the castle.

Why this matters (not skipped): Before today, our robot tester was checking **nothing** (the rooms had no alarms). Now the robots run **4 real checks** every single time. This was the **very first real test in the whole project** — a big first step. Everything passed: *"0 failed, 0 errors, of 4 tests."*

Build 4 — We taught the memory file who you are (and how to talk to you) 🧠

(Job #107 — *CLAUDE.md* team map + your talking style)

We wrote two important facts into the always-remember file:

1. **Who is who** on the team — **you are DEV-1** (the backend/builder boss), plus your two teammates and what each one builds.
2. **How you like me to talk** — like a **caveman** (short, fun words) — *but* with a strict rule: **caveman words never mean fewer facts**. I still explain everything, every check, every warning. Fun on the outside, complete on the inside.

Now every future chat already knows this, without you telling me again.

Build 5 — We made the store ready for MANY shop-owners (the big one) 🏢

(Job #3 = *P1-T02* — "Multi-Tenant Odoo Configuration & Secrets")

This is the one that turns "one shop" into "a building full of separate shops."

The clever choice you made: The store has an **instruction card** (a config file) that tells it how to behave. We already had a "**practice card**" that keeps *your* computer easy to use. The job wanted a "**real-world card**" with strict rules. If we changed the practice card into the strict one, **your store on your computer would break!**

So you picked the smart path: **keep your easy practice card, and make a SEPARATE real-world card**. Nothing on your computer changes. The real-world card waits, ready, for the day we go live.

What the real-world card says (all explained):

- **"Give each shop-owner their own door name → their own memory box."** So `clienta`'s door only ever opens `clienta`'s toys. (The grown-up word: `db_filter`.)
- **"Hide the list of all the shops."** So a nosy person can't see the other shop-owners' stores. We **proved** this: on the strict card, asking "show me all shops" gets a big **"NO, not allowed"** — but on the easy card it lists them. ✅
- **"Get ready for the bouncer."** (A helper called Nginx comes in the next job.)
- **"Use 4 workers"** so the store can serve 4 customers at once instead of 1.

Plus, we gave the store a doctor 🩺 (a "health check"). Two little doctors now keep asking each part *"are you feeling okay?"* — the memory box and the store itself. If one feels sick, we know right away. We also made the store **wait for the memory box to wake up fully** before it opens (no more opening before the brain is on).

Two bugs we caught by testing (before they could hurt us):

1. Odoo **hates** little notes written on the same line as a setting — it got confused and fell over. We moved every note to its own line. Fixed. ✅
2. One of *my own* checking commands lied and said "danger!" when there was none. I noticed, double-checked, and it was a false alarm. ✅

One thing I want you to NEVER forget (I put it in big letters in the report and the PR): The real-world card has a **pretend master-key** written in it right now. Before we ever go live for real, someone **must swap it for a real secret key**. It's safe today (the "list of shops" is already hidden), but it must be changed later.

The tidy-up we did too 🧹

Besides the 5 builds, we cleaned the house:

- **Opened a window to the memory box** so you can look inside with a tool called pgAdmin (through door **5433**, chosen so it doesn't fight the Postgres already on your laptop).
- **Threw away an old, unused memory box** (`ncollection_demo`) — it had none of our stuff in it. We kept the real one (`ncollection`).
- **Cleaned up old building tables** (branches) after their pieces were added.
- **Copied the workbench to the display shelf** (`main` = `develop`) so both are safe and matched.

Where everything stands right now ✅

Thing	Status
Jobs finished & merged today	5 (#104, #105, #106, #107, #108)
Jobs closed on the list today	3 (#103, #102, #3)
Jobs left to do	98 open
Building tables (branches)	Just develop + main , matched and clean

Robot checkers	All green on every build 
Your store on your computer	Works exactly the same — nothing broke

Two small rocks still in the road (your choice, no rush)

1. **The "auto-tick" doesn't tick by itself.** When we add a piece to the workbench (`develop`), the job on the to-do list doesn't cross itself off automatically — so I cross it off **by hand** each time (I did that today for #102 and #3). The one-time fix is to make `develop` the "main" table — **but you said not to do that for now**, so I keep ticking by hand. That's totally fine.
2. **The pretend master-key** in the real-world card must be swapped for a real one before we ever go live (a future job called P2-T08).

What comes next

The next job in your builder-chain is **#4 (P1-T03) — the bouncer (Nginx)**: the helper that stands at the front, sends each shop-owner to the right door, and blocks the "list of shops" page completely. In a fresh chat, just type: `/solve-issue 4` .

That's the whole day, start to finish — nothing left out. 